

Refatoracao de Testes e Deteccao de Test Smells

Qualidade e Testes de Software

Aluno

Rafael de Paiva Gomes

Matricula

691992

Pontifícia Universidade Católica de Minas Gerais

PUC Minas

Belo Horizonte, maio de 2026

1. Analise de Test Smells

Apos leitura detalhada do arquivo **userService.smelly.test.js**, foram identificados cinco Test Smells distintos. Os tres mais relevantes sao descritos a seguir.

#1 Logica Condicional no Teste (Conditional Logic)

Por que e um mau cheiro?

O teste percorre uma lista de usuarios com um **for** e, dentro do laco, utiliza um **if (!user.isAdmin)** para decidir quais verificacoes executar. Isso insere ramificacoes de controle dentro do proprio teste, tornando-o complexo e de dificil leitura.

Risco que representa:

Se a colecao estiver vazia ou a condicao nunca for satisfeita, os expects nao sao executados e o teste passa em verde sem validar nada, ocultando bugs silenciosamente. Alem disso, ao falhar, a mensagem de erro nao aponta claramente qual caso quebrou.

#2 Eager Test (Teste Faz Coisas Demais)

Por que e um mau cheiro?

O teste 'deve criar e buscar um usuario corretamente' realiza quatro acoes distintas em sequencia: cria o usuario, busca pelo ID, valida os dados de criacao e valida os dados de busca. Cada acao deveria ser um caso de teste independente.

Risco que representa:

Quando o teste falha nao e imediato saber se o problema esta na criacao ou na busca. O diagnostico de regressoes fica lento e a manutencao se torna custosa, pois qualquer mudanca em qualquer das operacoes pode quebrar o teste inteiro.

#3 Exception Handling Manual (Uso Incorreto de try/catch)

Por que e um mau cheiro?

Para verificar que uma excecao e lancada, o teste envolve a chamada em um bloco **try/catch** manual e coloca o **expect** dentro do bloco **catch**. O Jest disponibiliza o matcher **expect(...).toThrow(...)** exatamente para esse fim.

Risco que representa:

Se a funcao nao lancar excecao alguma, o bloco catch nunca e executado, os expects sao ignorados e o teste passa mesmo com um bug grave presente. E um falso positivo garantido: o teste 'verde' nao garante comportamento correto.

2. Processo de Refatoracao

O teste escolhido para demonstrar a refatoracao e o que verificava excecoes com **try/catch** manual, por concentrar tres smells ao mesmo tempo: Exception Handling Manual, Eager Test e ausencia do padrao **AAA (Arrange, Act, Assert)**.

Antes -Codigo Problematico (userService.smelly.test.js)

```
test('deve falhar ao criar usuario invalido', () => {  
  
  // sem Arrange explicito  
  
  try {  
  
    userService.createUser('', 'alice@email.com', 25);  
  
    userService.createUser('Alice', '', 25);  
  
    userService.createUser('Alice', 'alice@email.com', 16);  
  
  } catch (e) {  
  
    expect(e.message).toBe('Nome invalido');  
  
    expect(e.message).toBe('Email invalido');  
  
    expect(e.message).toBe('Usuario menor de idade');  
  
  }  
  
});
```

Depois -Codigo Refatorado (userService.clean.test.js)

```
describe('Criacao de usuarios', () => {  
  
  test('deve lancar erro ao criar usuario com nome vazio', () => {  
  
    // Arrange  
  
    const nome = '', email = 'alice@email.com', idade = 25;  
  
    // Act & Assert  
  
    expect(() => userService.createUser(nome, email, idade))  
  
      .toThrow('Nome invalido');  
  
  });  
  
});
```

```

test('deve lancar erro ao criar usuario com email vaziao', () => {

  // Arrange

  const nome = 'Alice', email = '', idade = 25;

  // Act & Assert

  expect(() => userService.createUser(nome, email, idade))

    .toThrow('Email invalido');

});

test('deve lancar erro ao criar usuario menor de idade', () => {

  // Arrange

  const nome = 'Alice', email = 'alice@email.com', idade = 16;

  // Act & Assert

  expect(() => userService.createUser(nome, email, idade))

    .toThrow('Usuario menor de idade');

});

});

```

Decisoes de Refatoracao

- 1. Separacao em testes atomicos:** cada cenario de erro virou um teste independente dentro do *describe* 'Criacao de usuarios'. Agora, ao falhar, o nome do teste identifica imediatamente qual regra de validacao esta quebrada.
- 2. Substituicao de try/catch por expect().toThrow():** o matcher nativo do Jest garante que o teste so passa se a execucao for de fato lancada. Se a funcao nao lancar nada, o teste falha corretamente, eliminando o risco de falso positivo silencioso.
- 3. Aplicacao do padrao AAA:** cada teste possui um bloco *Arrange* explicito com as variaveis de entrada, seguido do *Act* embutido no *expect* e do *Assert* com o matcher adequado. A estrutura torna a intencao do teste imediatamente legivel.

3. Relatório da Ferramenta (ESLint)

Após instalar as dependências **eslint** e **eslint-plugin-jest** e criar o arquivo **.eslintrc.json** conforme especificado no roteiro, o comando **npm run lint** foi executado sobre o arquivo **smelly**, produzindo a saída abaixo.

```
$ npm run lint

> test-smells-lab@1.0.0 lint
> eslint src/ test/

C:\...\test\userService.smelly.test.js

  44:9  error    Avoid calling `expect` conditionally  jest/no-conditional-expect
  46:9  error    Avoid calling `expect` conditionally  jest/no-conditional-expect
  49:9  error    Avoid calling `expect` conditionally  jest/no-conditional-expect
  73:7  error    Avoid calling `expect` conditionally  jest/no-conditional-expect
  77:3  warning  Tests should not be skipped           jest/no-disabled-tests
  77:3  warning  Test has no assertions                jest/expect-expect

6 problems (4 errors, 2 warnings)
```

A ferramenta identificou **4 erros** e **2 avisos**:

Linha	Regra	Descrição
44, 46, 49, 73	jest/no-conditional-expect	expect dentro de if/for (lógica condicional)
77	jest/no-disabled-tests	Teste ignorado com test.skip
77	jest/expect-expect	Teste sem nenhuma asserção

Como a ferramenta automatizou a detecção: ao invés de revisão manual linha a linha, o ESLint com o plugin *eslint-plugin-jest* analisou estaticamente todo o arquivo em milissegundos, apontando número de linha e regra violada com precisão. Isso tornaria inviável 'passar despercebido' um smell em bases de código grandes, e permite integrar a verificação em pipelines de CI/CD para bloquear merges com código de teste degradado.

4. Conclusão

Este trabalho demonstrou que a presença de Test Smells compromete diretamente a confiabilidade de uma suíte de testes, mesmo quando todos os casos retornam verde. Testes com lógica condicional, responsabilidades múltiplas ou tratamento incorreto de exceções criam uma falsa sensação de segurança, deixando bugs reais sem detecção.

A refatoração guiada pelo padrão **Arrange, Act, Assert** provou ser uma abordagem objetiva e eficaz: ao separar claramente as três fases do teste, o código ganha legibilidade e o diagnóstico de falhas se torna imediato. Cada teste passa a ter uma única responsabilidade e um nome que documenta o comportamento verificado.

A adoção de ferramentas de análise estática como o ESLint com *eslint-plugin-jest* amplifica esses benefícios ao escala. Identificar manualmente todos os smells em um repositório corporativo com centenas de arquivos de teste seria inviável; a ferramenta realiza essa varredura em segundos e pode ser integrada ao pipeline de CI/CD, impedindo que novos smells entrem na base de código principal.

Escrever testes limpos não é um custo adicional, mas um investimento na sustentabilidade do projeto. Uma suíte confiável reduz o tempo gasto em depuração, aumenta a coragem para refatorar o código de produção e documenta o comportamento esperado do sistema de forma viva e executável.